ENS RegistrarManager Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Stalin

Blckhv

Slavcheww

March 23, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **2**

6 Executive Summary **2**

6.0.1 Protocol Invariants 3

7 Findings **6**

7.1 Low Risk 6

7.1.1 Permissionless `withdrawAll` allows frontrunning a pending destination change 6

7.1.2 ETH-forwarding call via `execOnRegistrar` can be griefed by front-running with `withdrawAll` . 6

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

This ENS DAO governance proposal introduces two coordinated changes to the ENS treasury infrastructure:

1. `RegistrarManager` — a smart contract that acts as the owner of the ENS registrar controllers, enabling permissionless withdrawal of accumulated ETH registration fees and automatic routing of those funds to the ENS Endowment Safe
2. **Zodiac Roles Update** — an expansion of the Endowment Manager's operational permissions to allow limited ETH and USDC transfers from the Endowment Safe back to the ENS DAO Timelock for operational funding

5 Audit Scope

The audit scope was limited to:

```
- RegistrarManager.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the RegistrarManager smart contracts provided by ENS. In this period, a total of 2 issues were found.

The findings consist of 2 Low severity issues.

The first low is a DoS scenario on the `execOnRegistrar()` when intending to send ETH from the manager's balance to a registrar, a griefer can front-run with withdrawAll(), draining the balance first. The execOnRegistrar call then silently fails (success = false) without reverting, wasting the governance proposal and forcing a full new governance cycle to retry.

The second low is about a grief scenario when the Endowment Safe is compromised and the DAO needs to redirect ETH to a new destination via `setDestination`. A griefer can observe the pending governance transaction and repeatedly call withdrawAll() throughout the entire governance delay window — continuously draining any newly accumulated registrar ETH to the old (potentially compromised) destination before the update lands.

A formal verification suite implementing 19 properties & associated report was created and provided as an additional deliverable with the audit.

6.0.1 Protocol Invariants

As part of our protocol analysis we developed a number of proposed protocol invariants: 19 properties across 1 contract — 2 Invariants, 3 Parametric rules, 4 Access control rules, 7 Revert condition rules, 5 Integrity rules, 5 Sanity rules.

```
RegistrarManager

1) `destination` only changes via `setDestination`; no other function modifies it
2) `registrarCount` only increases via `addRegistrar` and only decreases via `removeRegistrar`; no other
↪  function modifies it
3) `_next[r]` for any address `r` only changes via `addRegistrar` or `removeRegistrar`; no other
↪  function modifies the linked list mapping
4) Any call to `addRegistrar` by a non-owner always reverts with `OwnableUnauthorizedAccount`
5) Any call to `removeRegistrar` by a non-owner always reverts with `OwnableUnauthorizedAccount`
6) Any call to `setDestination` by a non-owner always reverts with `OwnableUnauthorizedAccount`
7) Any call to `execOnRegistrar` by a non-owner always reverts with `OwnableUnauthorizedAccount`
8) `addRegistrar(address(0))` always reverts with `ZeroAddress`
9) `addRegistrar(address(0x1))` always reverts with `InvalidRegistrar`
10) `addRegistrar(r)` reverts with `RegistrarAlreadyExists` when `isRegistrar(r)` is already `true`
↪  before the call
11) `removeRegistrar(r)` reverts with `RegistrarNotFound` when `isRegistrar(r)` is `false` before the
↪  call
12) `setDestination(address(0))` always reverts with `ZeroAddress`
13) `setDestination(d)` reverts with `DestinationUnchanged` when `d` equals the current value of
↪  `destination`
14) `execOnRegistrar(r, value, data)` reverts with `RegistrarNotFound` when `isRegistrar(r)` is `false`
15) `addRegistrar` integrity: after a successful call with argument `r`, `isRegistrar(r)` returns `true`
↪  and `registrarCount` is exactly one greater than its value before the call
16) `removeRegistrar` integrity: after a successful call with argument `r`, `isRegistrar(r)` returns
↪  `false` and `registrarCount` is exactly one less than its value before the call
17) `setDestination` integrity: after a successful call with argument `newDest`, `destination` equals
↪  `newDest` and the previous destination address is no longer stored
18) `withdrawAll` integrity: after a successful call, the ETH balance of the `RegistrarManager` contract
↪  is zero regardless of how much ETH it held at the start of the call
19) `withdrawAll` is callable by any address (not just the owner) and succeeds whenever `destination`
↪  accepts ETH or the contract balance is zero
```

#	Category	CVL Name	CVL Type
1	State variable modification authorization	`onlySetDestinationChangesDestination`	Parametric
2	State variable modification authorization	`onlyAddRemoveChangesRegistrarCount`	Parametric
3	State variable modification authorization	`onlyAddRemoveChangesNextMapping`	Parametric
4	Access control	`addRegistrarOnlyOwner`	Access Control
5	Access control	`removeRegistrarOnlyOwner`	Access Control
6	Access control	`setDestinationOnlyOwner`	Access Control
7	Access control	`execOnRegistrarOnlyOwner`	Access Control
8	Revert conditions	`addRegistrarRevertsForZeroAddress`	Revert Condition

#	Category	CVL Name	CVL Type
9	Revert conditions	`addRegistrarRevertsForSentinel`	Revert Condition
10	Revert conditions	`addRegistrarRevertsIfAlreadyRegistered`	Revert Condition
11	Revert conditions	`removeRegistrarRevertsIfNotFound`	Revert Condition
12	Revert conditions	`setDestinationRevertsForZeroAddress`	Revert Condition
13	Revert conditions	`setDestinationRevertsIfUnchanged`	Revert Condition
14	Revert conditions	`execOnRegistrarRevertsIfNotRegistrar`	Revert Condition
15	Function post-condition integrity	`addRegistrarIntegrity`	Integrity
16	Function post-condition integrity	`removeRegistrarIntegrity`	Integrity
17	Function post-condition integrity	`setDestinationIntegrity`	Integrity
18	Function post-condition integrity	`withdrawAllZerosBalance`	Integrity
19	Permissionless access	`withdrawAllIsPermissionless`	Integrity

Summary

Project Name	RegistrarManager
Repository	dao-proposals
Commit	4b0348410dc7...
Fix Commit	e2f75840ee9b...
Audit Timeline	March 9th - March 11th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Informational	0
Gas Optimizations	0
Total Issues	2

Summary of Findings

[L-1] Permissionless `withdrawAll` allows frontrunning a pending destination change	Acknowledged

[L-2] ETH-forwarding call via `execOnRegistrar` can be griefed by front-running with `withdrawAll`	Resolved

7 Findings

7.1 Low Risk

7.1.1 Permissionless `withdrawAll` allows frontrunning a pending destination change

Description: `withdrawAll` is permissionless, meaning anyone can call it at any time to pull ETH from all registrars and forward the full contract balance to `destination`. If the owner needs to change `destination` via `setDestination` (e.g. because the Endowment Safe is compromised or its signers have lost access), a frontrunner can observe the pending `setDestination` transaction and call `withdrawAll` first, sending all accumulated ETH to the old `destination`.

Since the owner is the ENS DAO Timelock, the `setDestination` call goes through a governance delay. During that entire window, funds keep accumulating in the registrars and anyone can repeatedly call `withdrawAll` to drain them to the stale address.

RegistrarManager.sol#L176-L184

```
function withdrawAll() external {
    address registrar = _next[_HEAD];
    while (registrar != _HEAD) {
        bool success = _withdrawRegistrar(registrar);
        emit RegistrarWithdrawn(registrar, success);
        registrar = _next[registrar];
    }
    _forwardBalance();
}
```

Impact: If the Endowment Safe (`0x4F2083f5fBede34C2714aFfb3105539775f7FE64`) is compromised or its signers lose access, all ETH held across managed registrars can be forwarded there before the DAO can update `destination` through governance. The loss is the full balance at the time of the frontrun, and repeated calls can keep draining any newly accumulated ETH until the governance proposal finalizes.

Recommended Mitigation: Add an owner-only pause mechanism that stops `withdrawAll` from executing while a `destination` change is pending. Alternatively, bundle the `setDestination` call atomically with a `withdrawAll` in the same governance proposal so the old destination is never used after the decision to change it.

```
bool public paused;

function setPaused(bool _paused) external onlyOwner {
    paused = _paused;
}

function withdrawAll() external {
    require(!paused, "paused");
    // ...existing logic
}
```

Blockful: Acknowledged. The risk is conditional on the Endowment Safe being compromised, and the current destination being DAO-controlled mitigates the concern.

7.1.2 ETH-forwarding call via `execOnRegistrar` can be griefed by front-running with `withdrawAll`

Description: `execOnRegistrar` allows the owner to make arbitrary calls to a managed registrar, optionally attaching ETH from the RegistrarManager's own balance via the `value` parameter:

```
// RegistrarManager.sol line 199-213
function execOnRegistrar(
    address registrar,
    uint256 value,       // ← ETH drawn from address(this).balance
    bytes calldata data
)
```

```
    external onlyOwner returns (bool success, bytes memory result)
{
    if (!isRegistrar(registrar)) revert RegistrarNotFound(registrar);
    (success, result) = registrar.call{ value: value }(data);  // ← silent failure if balance < value
    emit RegistrarCall(registrar, value, data, success);
}
```

Because `withdrawAll` is **permissionless**, a griefer can front-run the owner's pending `execOnRegistrar` transaction with a `withdrawAll()` call, draining the RegistrarManager's ETH balance to `destination` before the governance transaction lands. When `execOnRegistrar` then executes, `address(this).balance < value` and the low-level call silently fails — returning `success = false` without reverting.

Attack Scenario:

1. The DAO passes a governance proposal calling `execOnRegistrar(registrar, 5 ether, data)` to fund or interact with a registrar with ETH
2. The RegistrarManager holds 5 ETH (accumulated from prior registrar withdrawals)
3. A griefer sees the pending governance transaction in the mempool and front-runs with `withdrawAll()`
4. `withdrawAll()` drains the 5 ETH from RegistrarManager to the Endowment Safe
5. `execOnRegistrar` executes — `address(this).balance == 0`, the call to the registrar fails silently with `success = false`
6. The governance action is wasted; a new proposal must be created and wait through the full governance delay

Impact: No ETH is stolen — funds are forwarded to the Endowment Safe (the intended long-term destination). However, the Timelock's intended governance action fails silently, and recovery requires creating and waiting through a full new governance proposal cycle. This attack can be repeated indefinitely at near-zero cost to the griefer (only gas).

Recommended Mitigation: There are a couple of alternatives to mitigate this issue:

1. When the owner intends to send ETH to a registrar via `execOnRegistrar`, the ETH should be included directly as `msg.value` in the governance transaction rather than relying on the contract's accumulated balance. Add a `payable` modifier to `execOnRegistrar` and use `msg.value` for the call.
2. Similar recommendation as in *Permissionless `withdrawAll` allows frontrunning a pending destination change* . Add a pauser modifier that in case of facing a DoS, the `withdrawAll` function can be paused.
3. Add an access modifier to `withdrawAll` to allow only authorized entities, fully preventing any possibilities of frontrun.

Blockful: Fixed in commit e2f7584.

Cyfrin: Verified. `execOnRegistrar` function no longer forwards ETH — calls registrar with zero value.